

CampusBuddy

Full Application Description

Workflow · Data Flow · Navigation Map · UI State Management

Application	CampusBuddy — Student Partner Finder
Platform	Android (Kotlin) / Jetpack Compose
Purpose	A mobile app where verified college students find study partners, project teammates, event buddies, and placement prep partners. Students post requests, respond to others, get matched, and chat.
Total Screens	19 screens across Auth, Setup, Main (BottomNav), and Detail layers
Total Sections	23 sections covering every screen, data model, enum, flow, navigation route, and UI rule
Document Type	Complete Developer Specification — Ready for Android Implementation



TABLE OF CONTENTS

01	Screens List — All 19 screens grouped by layer
02	Data Models — 8 Kotlin data classes with all fields
03	Enum / State Values — 7 enums with full value descriptions
04	Auth Flows — App Launch · Login · Signup · Email Verification · Forgot Password
05	Profile Setup — First-time onboarding flow with chip selections
06	Home Screen — Components, lifecycle, data loading, interactions
07	Requests Screen — Browse Requests & My Requests tabs
08	Create Request Screen — Form fields, validation, execution flow
09	Request Details Screen — Responder view with join logic
10	My Request Details Screen — Creator view with accept/reject/close logic
11	Partner Profile Screen — Compatibility card, action controls
12	Chats Screen — Conversation list, badges, filtering
13	Chat Screen — Real-time lifecycle, bubbles, message dispatch
14	Profile Screen — My Profile tab with streak, reliability, logout
15	Edit Profile Screen — Modification flow
16	Notifications Screen — Type-based routing map
17	Streaks & Reliability — Calculation formulas and logic
18	Settings Screen — Notifications, account, app sections
19	Report User Screen — Reason dropdown, submit flow
20	Matches Screen — Match list with open chat
21	Bottom Navigation Bar — 4-tab persistent shell
22	Complete Navigation Map — Every screen-to-screen route
23	General UI Rules — Loading, empty, error, button, confirmation, navigation standards

The application contains **19 screens** organized across four distinct architectural layers. Each layer determines how the screen is placed in the navigation stack and when it is shown to the user.

AUTH SCREENS

SplashScreen	Entry point. Checks session and routes user to the correct destination on app launch.
LoginScreen	Allows existing users to authenticate with college email and password.
SignupScreen	New user registration with full college identity fields.
EmailVerificationScreen	6-digit OTP verification to confirm college email ownership.
ForgotPasswordScreen	Sends a password reset link to the registered college email.

SETUP SCREENS

ProfileSetupScreen	Shown once after email verification. User selects interests, skills, availability, and goals via chip UI.
OnboardingScreen	Shown once after profile setup. Introduces the app features to the new user.

MAIN SCREENS (BottomNavigation)

HomeScreen — Tab 1	Dashboard with greeting, streak badge, reliability badge, recommended partners, and open requests.
RequestsScreen — Tab 2	Two-tab screen: Browse Requests (all college requests) and My Requests (creator view).
ChatsScreen — Tab 3	List of all active conversations with real-time unread badge on BottomNav tab.
ProfileScreen — Tab 4	Full user profile with streak card, reliability card, edit, settings, and logout.

DETAIL SCREENS (Stack-pushed)

CreateRequestScreen	Form to post a new partner request with type, title, range, availability, and needed count.
RequestDetailsScreen	Responder view of a request with creator card and Request to Join button.
MyRequestDetailsScreen	Creator view of their own request with accept/reject controls per response.
PartnerProfileScreen	Full profile of another student with compatibility card and optional chat button.
ChatScreen	Real-time 1:1 or group chat screen with message bubbles and send functionality.

MatchesScreen	Lists all accepted matches with direct Open Chat button.
NotificationsScreen	All notifications with type-based icon routing to relevant screens.
EditProfileScreen	Editable version of ProfileSetupScreen with save to DB.
SettingsScreen	Notification toggles, change password, delete account, privacy/terms links.
ReportUserScreen	Reports a user with reason dropdown and optional description.

Eight Kotlin **data classes** define the complete data layer of CampusBuddy. Each model maps directly to a database table. Below are the full field definitions with types, nullability, defaults, and constraints.

UserProfile

```
data class UserProfile(
    val id: String // Unique user identifier (auth UID)
    val fullName: String // User's full real name
    val collegeEmail: String // Must be college-domain email
    val registrationNumber: String // Unique per college, validated on signup
    val collegeName: String // College name string
    val department: String // e.g. Computer Science, Mechanical
    val year: String // e.g. 1st, 2nd, 3rd, 4th
    val bio: String? // Optional short bio
    val profilePhotoUrl: String? // Optional cloud storage URL
    val interests: List // e.g. ["DSA", "ML", "Photography"]
    val skills: List // e.g. ["Kotlin", "Python", "Figma"]
    val availability: List // e.g. ["morning", "evening", "weekends"]
    val goals: List // e.g. ["Crack Placements", "Win Hackathons"]
    val isVerifiedStudent: Boolean // default: false → true after OTP
    val status: UserStatus // UNVERIFIED | VERIFIED | ALUMNI | BANNED
    val role: UserRole // USER | ADMIN
    val currentStreak: Int // default: 0, increments on daily login
    val reliabilityScore: Float // default: 0.0, recalculated on match/msg
    val createdAt: Long // Unix timestamp of account creation
)
```

PartnerRequest

```
data class PartnerRequest(
    val id: Long // Auto-generated primary key
    val creatorId: String // FK → UserProfile.id
    val type: RequestType // STUDY|PROJECT|HACKATHON|EVENT|PLACEMENT
    val title: String // Min 5 chars, not empty
    val description: String? // Optional extended description
    val subjectOrTopic: String? // Optional subject/topic label
    val preferredRange: RangeType // CLASS|DEPARTMENT|COLLEGE|NEARBY|ONLINE
    val preferredSkills: List // Optional skill filter chips
    val availability: String? // Optional availability string
    val neededCount: Int // 1-10, how many partners needed
    val acceptedCount: Int // default: 0, increments on each accept
    val status: RequestStatus // OPEN|FULL|CLOSED|CANCELLED
    val createdAt: Long // Unix timestamp
)
```

PartnerResponse

```
data class PartnerResponse(  
    val id: Long // Auto-generated primary key  
    val requestId: Long // FK → PartnerRequest.id  
    val responderId: String // FK → UserProfile.id  
    val status: ResponseStatus // PENDING|ACCEPTED|REJECTED  
    val createdAt: Long // Unix timestamp  
)
```

Match

```
data class Match(  
    val id: Long // Auto-generated primary key  
    val user1Id: String // FK → UserProfile.id (creator)  
    val user2Id: String // FK → UserProfile.id (responder)  
    val requestId: Long // FK → PartnerRequest.id  
    val matchScore: Float // Computed compatibility score  
    val createdAt: Long // Unix timestamp of match creation  
)
```

Conversation

```
data class Conversation(  
    val id: Long // Auto-generated primary key  
    val isGroup: Boolean // true for group chats  
    val createdBy: String // FK → UserProfile.id  
    val memberIds: List // All participant user IDs  
    val createdAt: Long // Unix timestamp  
)
```

Message

```
data class Message(  
    val id: Long // Auto-generated primary key  
    val conversationId: Long // FK → Conversation.id  
    val senderId: String // FK → UserProfile.id  
    val content: String // Message text content  
    val isRead: Boolean // default: false → true on recipient open  
    val createdAt: Long // Unix timestamp for ordering  
)
```

Notification

```
data class Notification(  
    val id: Long // Auto-generated primary key  
    val userId: String // FK → UserProfile.id (recipient)  
    val type: NotificationType // Determines icon and navigation target  
    val title: String // Short notification title  
    val body: String // Notification body text  
    val isRead: Boolean // default: false  
    val referenceId: String? // requestId or conversationId depending on type  
    val createdAt: Long // Unix timestamp  
)
```

ActivityLog

```
data class ActivityLog(  
    val id: Long // Auto-generated primary key  
    val userId: String // FK → UserProfile.id  
    val activityDate: String // Date only string (yyyy-MM-dd)  
    val activityType: String // LOGIN | MESSAGE | MATCH | SESSION  
)
```

Seven enumerations define every possible state across the application. These are used for type safety, UI state management, conditional logic, and database persistence.

enum class UserStatus

UNVERIFIED	User has created an account but has not yet completed email OTP verification. Cannot post requests or message others.
VERIFIED	Confirmed college student. Full access to all app features.
ALUMNI	User has graduated or left college. May have limited access.
BANNED	Admin-banned account. Redirected to LoginScreen with suspension message on app launch.

enum class UserRole

USER	Normal student account with standard permissions.
ADMIN	Admin account. Can moderate reports, ban users, and view all content.

enum class RequestType

STUDY	Request to find a study partner for exams or coursework.
PROJECT	Request to find teammates for an academic or personal project.
HACKATHON	Request to form a hackathon team.
EVENT	Request to find a buddy for college events.
PLACEMENT	Request to find a partner for placement/interview preparation.

enum class RangeType

CLASS	Visible only to students in the same class section.
DEPARTMENT	Visible to entire department.
COLLEGE	Visible to the entire college.
NEARBY	Visible to students in nearby colleges.
ONLINE	No location restriction; visible to all verified users.

enum class RequestStatus

OPEN	Request is active and accepting new responses.
FULL	acceptedCount has reached neededCount. No more responses accepted. closeRequestButton hides.
CLOSED	Creator manually closed the request. No longer accepting responses.
CANCELLED	Creator cancelled the request before completion.

enum class ResponseStatus

PENDING	Response submitted, awaiting creator's accept or reject decision.
ACCEPTED	Creator accepted this response. Match + Conversation created. REQUEST_ACCEPTED notification sent.
REJECTED	Creator rejected this response. REQUEST_REJECTED notification sent to responder.

enum class NotificationType

REQUEST_RECEIVED	Sent to request creator when a new response is submitted. Icon: Person. Routes to MyRequestDetailsScreen.
REQUEST_ACCEPTED	Sent to responder when creator accepts. Icon: Checkmark. Routes to ChatScreen.
REQUEST_REJECTED	Sent to responder when creator rejects. Icon: X. Routes to RequestsScreen (My Requests tab).
NEW_MESSAGE	Sent to recipient when a new chat message is received. Icon: Chat. Routes to ChatScreen.
STREAK_MILESTONE	Sent to user when streak hits 7, 14, or 30 days. Icon: Fire. Routes to ProfileScreen.

Flow: App Launch — SplashScreen

Screen	SplashScreen
Initial State	loading
UI Changes	Spinner shows while checking session → hidden when navigation begins
Side Effects	Session check, UserProfile fetch, status evaluation

Step 1 Display Logo Show app logo and name for approximately 2 seconds.

Step 2 Check Session Query auth storage for an existing user session.

Step 3a Session Exists Fetch UserProfile for the current user ID.

Step 3b No Session Navigate directly to LoginScreen.

Step 4a Profile BANNED Navigate to LoginScreen. Show toast: "Your account has been suspended."

Step 4b Profile Complete name + department + year present → Navigate to HomeScreen.

Step 4c Profile Incomplete Missing required fields → Navigate to ProfileSetupScreen.

Flow: Login — LoginScreen

Screen	LoginScreen
Initial State	idle
Data Input	college_email, password
Data Output	User session + UserProfile object
Components	collegeEmailField · passwordField · loginButton · errorText · forgotPasswordLink · goToSignupLink
Field States	empty typing valid error
Button States	enabled loading disabled

Step 1 Email Input User types college email → collegeEmailField state: empty → typing.

Step 2 Password Input User types password → passwordField state: empty → typing.

Step 3 Tap Login loginButton: loading (spinner shown, button disabled). Keyboard dismisses.

Step 4a Success Log LOGIN to ActivityLog. Update streak (check yesterday in ActivityLog; if exists → increment; else reset to 1). Navigate to HomeScreen, clearing entire auth backstack.

Step 4b Wrong Credentials loginButton: enabled. errorText visible: "Invalid email or password." Both fields show error border/color.

Step 4c Network Error loginButton: enabled. errorText: "Network error. Please check your connection."

Flow: Signup — SignupScreen

Screen	SignupScreen
Data Input	fullName, collegeEmail, registrationNumber, collegeName, department, year, password
Data Output	UserProfile row (status=UNVERIFIED, isVerifiedStudent=false)

Components

fullNameField · collegeEmailField · registrationNumberField · collegeNameField · departmentField · yearDropdown · passwordField · confirmPasswordField · signUpButton · errorText

Validation Rules (all must pass before signUpButton → loading):

- **fullName**: not empty
- **collegeEmail**: valid email format
- **registrationNumber**: not empty AND unique (checked against DB)
- **collegeName**: not empty
- **department**: not empty
- **year**: selected from dropdown
- **password**: minimum 8 characters
- **confirmPassword**: must exactly match password field
- Any failure → show inline error below that specific field. signUpButton remains enabled.

Step 1 Fill Fields User fills all fields. Each field cycles: empty → typing → valid / error.

Step 2 Tap Create Account Trigger full field validation (see rules above).

Step 3a All Valid signUpButton: loading → Create auth account → Create UserProfile row (status=UNVERIFIED) → Navigate to EmailVerificationScreen.

Step 3b Email In Use signUpButton: enabled. errorText: "This email is already registered. Please log in."

Step 3c Reg Number In Use registrationNumberField shows error. errorText: "This registration number is already registered."

Flow: Email Verification — EmailVerificationScreen

Screen	EmailVerificationScreen
Data Input	6-digit OTP code
Data Output	UserProfile.status → VERIFIED, UserProfile.isVerifiedStudent → true
Components	infoText (We sent a code to {email}) · otpField (6-digit) · verifyButton · resendButton · errorText
Button States	verifyButton: enabled loading resendButton: enabled countdown

Step 1 Screen Opens resendButton shows countdown timer (e.g. Resend in 60s). Timer ticks down visually.

Step 2 Countdown Ends resendButton becomes enabled and label changes to "Resend code".

Step 3 Enter OTP User types OTP → otpField auto-fills digit by digit.

Step 4 Tap Verify verifyButton: loading.

Step 5a Correct OTP Update UserProfile: isVerifiedStudent=true, status=VERIFIED. Navigate to ProfileSetupScreen.

Step 5b Wrong OTP verifyButton: enabled. otpField: red border (error state). errorText: "Incorrect code. Please try again."

Step 5c OTP Expired errorText: "Code has expired. Please tap Resend." otpField clears automatically.

Flow: Forgot Password — ForgotPasswordScreen

Screen	ForgotPasswordScreen
Components	collegeEmailField · sendResetButton (enabled loading) · successText (hidden visible) · errorText (hidden visible)

Step 1 Enter Email User types their registered college email address.

Step 2 Tap Send Reset Link sendResetButton: loading.

Step 3a Email Found sendResetButton: enabled. successText visible: "Password reset link sent to your email."

Step 3b Email Not Found sendResetButton: enabled. errorText: "No account found with this email."

This screen is shown **exactly once** immediately after successful email verification. It collects personalization data used for matching algorithms and feed recommendations.

Screen	ProfileSetupScreen
Trigger	Shown once after email OTP verification
Data Input	bio, interests[], skills[], availability[], goals[], profilePhoto
Data Output	Updates existing UserProfile row (same ID) with all selected values

All Chip Options:

profilePhotoUpload	state: empty uploading uploaded — Cloud upload with progress indicator
bioField	Optional free-text field for short self-description
interestChips	DSA · Web Dev · ML · AI · UI/UX · Robotics · Competitive Programming · Public Speaking · Photography · Music · Dance · Sports · Entrepreneurship · Design
skillChips	Python · Java · Kotlin · JavaScript · C++ · SQL · Git · Figma · Communication · Leadership · Research
availabilityChips	Morning · Afternoon · Evening · Night · Weekdays · Weekends
goalChips	Crack Placements · Pass Exams · Build Projects · Win Hackathons · Learn New Skills · Participate in Events
saveButton	Disabled until minimum selections: ≥ 2 interests AND ≥ 1 availability

Step 1 Tap Chips Each tapped chip toggles: outlined (unselected) ↔ filled/colored (selected). saveButton enables only after minimum selections met.

Step 2 Tap Save & Continue saveButton: loading.

Step 3 Success Update UserProfile row with all selected data. Navigate to OnboardingScreen or HomeScreen.

Screen	HomeScreen
Access	BottomNavigation Tab 1 (house icon)
Type	Read-only display screen — no write operations

Components

greetingText	Displays "Hi, {name}" using UserProfile.fullName
streakBadge	Shows "Current streak: {X} days". Hidden if streak == 0
reliabilityBadge	Shows "{X}% Reliable" from UserProfile.reliabilityScore
notificationIcon	no-dot state normally. Red dot visible if unread notification count > 0
createRequestButton	FAB or prominent button. Tap → CreateRequestScreen
recommendedPartnersSection	Horizontal list of partner cards (limit 5, sorted by match score)
openRequestsSection	Vertical list of request cards (limit 10, from user's college)

Lifecycle & Data Loading

- On screen open: all data loads asynchronously in parallel.
- Fetch UserProfile for current user.
- Fetch top 5 recommended partners based on shared interests, skills, goals, availability (matching score algorithm).
- Fetch 10 most recent OPEN PartnerRequests from the user's college.
- Fetch unread notification count → if count > 0, show red dot on notificationIcon.
- Show grey skeleton placeholder cards while loading → replace with real data on completion.

Recommended Partner Card

- partnerName
- departmentAndYear
- interestChips (first 3 matching interests)
- compatibilityBadge (e.g. 87% Match)
- reliabilityIcon (e.g. 92%)
- viewButton → tap anywhere on card: Navigate to PartnerProfileScreen

Open Request Card

- requestTitle
- requestType badge (color-labeled by type)
- subjectOrTopic text
- creatorName + department
- neededCount text
- postedTime (relative, e.g. 2h ago) → tap: Navigate to RequestDetailsScreen with requestId

Screen	RequestsScreen
Access	BottomNavigation Tab 2 (list icon)
Layout	Two-tab layout: Browse Requests My Requests
Components	tabBar · filterBar (Browse tab only) · requestList · createRequestFAB (always visible)
List States	loading loaded empty error

Tab 1 — Browse Requests

- On open: Fetch all OPEN PartnerRequests from user's college. Apply current filter values. Show skeleton loaders.
- Empty state: illustrative image + text "No open requests found. Create one!"
- Filter change (typeFilter or rangeFilter dropdown): requestList immediately re-fetches/re-filters → briefly loading → loaded.
- Request Card displays: title, typeBadge, subjectOrTopic, creatorName + department, neededCount, postedTime.
- Tap card → Navigate to RequestDetailsScreen.

Tab 2 — My Requests

- On open: Fetch PartnerRequests where creatorId == currentUserId. Sort by createdAt descending.
- Empty state: "You have not created any requests yet."
- My Request Card displays: title, typeBadge, statusBadge (OPEN→green, FULL→blue, CLOSED→grey, CANCELLED→red), responseCount, acceptedCount.
- Tap card → Navigate to MyRequestDetailsScreen.

Screen	CreateRequestScreen
Trigger	FAB on HomeScreen or RequestsScreen
Data Output	New PartnerRequest row saved to partner_requests table

Form Fields & Component States

typeDropdown	unselected selected — RequestType values: STUDY, PROJECT, HACKATHON, EVENT, PLACEMENT
titleField	empty typing valid error — min 5 characters required
descriptionField	empty typing valid — optional but recommended
subjectField	empty typing valid — optional subject/topic label
rangeDropdown	unselected selected — RangeType values: CLASS, DEPARTMENT, COLLEGE, NEARBY, ONLINE
availabilityChips	multi-select — Morning, Afternoon, Evening, Night, Weekdays, Weekends
preferredSkillsChips	multi-select, optional — skill tags for filtering responses
neededCountStepper	number stepper, range: 1 to 10 — how many partners are needed
postButton	disabled until validation passes enabled loading
cancelButton	always enabled — pops navigation without saving

Validation Rules (postButton enabled when ALL pass):

- typeDropdown is selected
- titleField is not empty AND has minimum 5 characters
- rangeDropdown is selected
- At least 1 availability chip is selected

Step 1 Enter Details User fills form. Each field toggles through states as user interacts. Stepper increments/decrements between 1–10.

Step 2 Tap Post Request postButton: loading. Keyboard dismisses.

Step 3a Success PartnerRequest saved with status=OPEN, acceptedCount=0, createdAt=now. Navigate back to RequestsScreen (My Requests tab). New request appears at top of list.

Step 3b Error postButton: enabled. errorSnackBar: "Could not post request. Please try again."

Screen	RequestDetailsScreen
Trigger	Responder taps a request from Browse Requests list or HomeScreen open requests
Data Input	requestId passed via navigation argument
Data Output	PartnerResponse row (status=PENDING) + Notification to creator (REQUEST_RECEIVED)

Displayed Information

- Full request title and type
- Subject or topic (if provided)
- Description (if provided)
- Preferred range and availability
- Needed count (X partners needed)

Creator Card

- Creator name and profile photo
- Department and year
- Reliability score
- Current streak
- Verified student badge
- viewProfileButton → Navigate to PartnerProfileScreen

Respond Button Logic

No existing response	respondButton shows "Request to Join" — enabled
Response already sent	respondButton shows "Already Requested" — disabled

Step 1 Screen Opens Load PartnerRequest data and creator's UserProfile. Check if currentUser has an existing PartnerResponse for this requestId.

Step 2 Tap Request to Join Button state → loading.

Step 3a Success Button: disabled, text → "Requested ✓". successSnackBar: "Your request has been sent." PartnerResponse row created (status=PENDING). Notification (REQUEST_RECEIVED) pushed to creator.

Step 3b Error Button re-enables. errorSnackBar shown.

<i>Screen</i>	MyRequestDetailsScreen
<i>Trigger</i>	Creator taps one of their own requests in My Requests tab
<i>Components</i>	Request info · statusBadge · acceptedCountText · closeRequestButton · responsesSection
<i>responsesSection</i>	List of all PartnerResponse rows for this request, each showing responder info, action buttons, and status label

Accept Response Logic

- Creator taps Accept → confirmation dialog appears.
- If confirmed: acceptButton enters loading state.
- After DB save: PartnerResponse.status → ACCEPTED.
- Buttons on that row disappear. Green "Accepted" label appears.
- acceptedCountText increments.
- A Match row is created in DB (user1Id=creatorId, user2Id=responderId, requestId).
- A Conversation is created in DB with both users as members.
- Notification (REQUEST_ACCEPTED) pushed to responder.
- If acceptedCount == neededCount: PartnerRequest.status → FULL. closeRequestButton hides. Open Chat button appears on that response row.

Reject Response Logic

- Creator taps Reject → no confirmation required.
- PartnerResponse.status → REJECTED.
- Buttons disappear. Red "Rejected" label appears.
- Notification (REQUEST_REJECTED) dispatched to responder.

Close Request Logic

- Creator taps closeRequestButton (only visible when status=OPEN) → confirmation dialog.
- If confirmed: PartnerRequest.status → CLOSED.
- statusBadge updates to grey. closeRequestButton hides.

<i>Data Flow</i>	Updates PartnerResponse.status · Writes Match row · Writes Conversation row · Creates notification logs
------------------	---

<i>Screen</i>	PartnerProfileScreen
<i>Trigger</i>	Tapping any student card (from HomeScreen, MyRequestDetailsScreen, ChatScreen top bar, or direct navigation)
<i>Data Input</i>	partnerUserId passed as navigation argument
<i>Data Output</i>	Read-only. No writes except via reportUserButton flow.

Displayed Components

- Profile photo (full size or placeholder)
- Verification badge (Verified Student indicator)
- Status badge (VERIFIED / ALUMNI)
- Full name, college name, department, year
- Bio (if set)
- Read-only interest chips
- Read-only skill chips
- Streak and reliability indicators

Compatibility Card

Overall Match %	Computed from shared interests, availability, goals, and subject overlap
Same Interests	Indicator row showing shared interest tags
Same Availability	Indicator row for overlapping time slots
Same Goals	Indicator row for matching goal chips
Same Subject	Indicator row for subject/topic overlap

Action Controls

Already Matched	Open Chat button shown — navigates directly to ChatScreen for their shared Conversation
Not Matched	No direct action button. Matching only occurs through formal request flow.
reportUserButton	Always available in overflow menu or footer → Navigate to ReportUserScreen

Screen	ChatsScreen
Access	BottomNavigation Tab 3 (chat bubble icon)
Components	chatList (loading loaded empty) · searchField · unread badge on BottomNav
List States	loading loaded empty

- On open: Fetch all Conversations where currentUser is in memberIds. For each conversation, fetch the last Message object.
- Show skeleton cards while loading.
- Empty state: "No chats yet. Accept a partner to start chatting."
- Conversation item displays: partnerName (or group name), lastMessagePreview (first 40 characters), timeAgo, unreadDot.
- Tap conversation item → Navigate to ChatScreen. On return: chatList refreshes, unreadDot hides for that item.
- Typing in searchField filters chatList by partner name in real time.
- BottomNav Tab 3 unread badge: updates dynamically as new messages arrive. Clears when all active chats are read.

<i>Screen</i>	ChatScreen
<i>Trigger</i>	Tap on conversation in ChatsScreen, or Open Chat button after match acceptance
<i>Components</i>	Top bar · scrollable messagesList · messageInputField · sendButton
<i>sendButton</i>	Disabled when messageInputField is empty — enables on any input

Top Bar Components

- Back arrow (pops navigation)
- Partner or group name
- Online status indicator
- Overflow menu (profile link, report option)

Real-Time Lifecycle

- On open: Fetch all historical messages for this conversationId ordered by createdAt ASC.
- Mark all unread messages (isRead=false) as read immediately on open.
- Establish Realtime subscription for new Message rows targeting this conversationId.
- Auto-scroll to bottom on initialization and on every new message received.

Message Bubbles

Current User	Right-aligned. Colored background. Shows timestamp below. Single tick (sent) → double tick (read, if enabled).
Partner	Left-aligned. Grey background. Shows timestamp below.
New Messages	Appended at bottom in real time. Auto-scroll triggered.

Message Dispatch Flow

- User types message and taps sendButton.
- messageInputField clears immediately.
- New Message row written to DB (conversationId, senderId, content, isRead=false, createdAt=now).
- NEW_MESSAGE notification fired to all other members of the conversation.
- ActivityLog row inserted: activityType=MESSAGE, activityDate=today.

Screen	ProfileScreen
Access	BottomNavigation Tab 4 (person icon)

Displayed Components

Profile Photo	Full display with upload state from last edit
Verification Badge	Shows if isVerifiedStudent == true
Status Badge	Shows VERIFIED or ALUMNI with color coding
Full Name & College	fullName, collegeName, department, year
streakCard	currentStreak value with fire icon. Hidden if streak == 0.
reliabilityCard	reliabilityScore as percentage with indicator color
Category Chips	Read-only interest, skill, availability, goal chips
editProfileButton	Navigates to EditProfileScreen
settingsButton	Navigates to SettingsScreen
logoutButton	Triggers confirmation dialog

Operations

- Screen loads with skeleton blocks → replaced by UserProfile data.
- Edit Profile: Navigate to EditProfileScreen. On return, all profile elements reload and update.
- Logout: Confirmation dialog with Cancel and Confirm. On Confirm: session cleared, navigate to LoginScreen with entire backstack cleared.

<i>Screen</i>	EditProfileScreen
<i>Trigger</i>	editProfileButton on ProfileScreen
<i>Structure</i>	Same fields as ProfileSetupScreen + active profilePhotoUpload state indicator
<i>saveButton</i>	Disabled until at least one field value changes from current saved state
<i>On Save</i>	Form validation → write changes to UserProfile row in DB → navigate back to ProfileScreen (backstack pop) → ProfileScreen reloads with fresh data

Screen	NotificationsScreen
Trigger	Tap notificationIcon on HomeScreen or ProfileScreen
Components	notificationList · markAllReadButton (only visible when unread items exist)

Notification Type Routing Map

REQUEST_RECEIVED	Icon: Person — Routes to MyRequestDetailsScreen. Creator views the new response.
REQUEST_ACCEPTED	Icon: Checkmark — Routes to ChatScreen. Responder opens the new chat.
REQUEST_REJECTED	Icon: X — Routes to RequestsScreen (My Requests Tab). Responder sees request status.
NEW_MESSAGE	Icon: Chat bubble — Routes to ChatScreen. User opens the new message.
STREAK_MILESTONE	Icon: Fire — Routes to ProfileScreen. User sees their streak achievement.

Behavior

- Tapping an individual notification: sets isRead=true, updates background styling (read vs unread visual), routes user to the relevant screen.
- Tapping markAllReadButton: DB batch update sets all notification rows to isRead=true. All notification backgrounds update simultaneously.

Streak Calculation Logic

- A streak increments by 1 each day the user performs at least one transaction (login, message, match, or session).
- Streak calculation runs on every login event.
- Query ActivityLog for any entry where `userId==currentUserId AND activityDate == yesterday`.
- Entry found → `UserProfile.currentStreak += 1`
- No entry found → `UserProfile.currentStreak = 1` (reset)
- Write new ActivityLog row: `activityType=LOGIN, activityDate=today`.

Streak Milestones

7 Days	Trigger STREAK_MILESTONE notification: title = "7-day streak!", fire icon, routes to ProfileScreen
14 Days	Trigger STREAK_MILESTONE notification: title = "14-day streak!"
30 Days	Trigger STREAK_MILESTONE notification: title = "30-day streak!"

Reliability Score Calculation

The reliability score measures how often a user follows through after being matched. It is recalculated dynamically and persisted to `UserProfile.reliabilityScore`.

```
totalInteractions = Count(Match rows where user1Id OR user2Id == currentUserId)
```

```
completedInteractions = Count(Match rows with at least 1 Message exchange in linked Conversation)
```

```
reliabilityScore = (completedInteractions / totalInteractions) x 100
```

- Recalculated in the repository layer when a new Match row is created.
- Recalculated in the repository layer when a new Message row is successfully committed.
- Result persisted immediately back to `UserProfile.reliabilityScore`.
- Displayed on ProfileScreen `reliabilityCard` and on partner cards in HomeScreen.

Screen	SettingsScreen
Trigger	settingsButton on ProfileScreen

Notification Settings

- Toggle: New messages — enable/disable NEW_MESSAGE notifications
- Toggle: Request responses — enable/disable REQUEST_RECEIVED / REQUEST_ACCEPTED / REQUEST_REJECTED notifications
- Toggle: Streak milestones — enable/disable STREAK_MILESTONE notifications

Account Section

changePasswordButton	Opens password change flow (email link or in-app form)
deleteAccountButton	Demands confirmation dialog. On confirm: wipes all user rows from DB, invalidates auth session, navigates to LoginScreen. Irreversible.

App Section

- Privacy Policy — tappable text link
- Terms of Service — tappable text link
- App Version — current version string (read-only)

Screen	ReportUserScreen
Trigger	reportUserButton in PartnerProfileScreen overflow menu or footer
Data Output	Report log row inserted to DB with status=OPEN

Components

Target User Name	Read-only string — name of the user being reported
reasonDropdown	Strict selection — Spam Harassment Fake Account Inappropriate Content Other
descriptionField	Optional free-text paragraph for additional context
submitButton	Disabled until a reason is selected from dropdown

Step 1 Screen Opens Target user name displayed as read-only. All other fields empty.

Step 2 Select Reason submitButton becomes enabled when a reason is chosen from dropdown.

Step 3 Optional Description User may optionally add a description paragraph.

Step 4 Tap Submit Insert report log to DB (status=OPEN). Pop navigation backward. Show confirmation toast snackbar.

<i>Screen</i>	MatchesScreen
<i>Access</i>	From HomeScreen (see all matches) or ProfileScreen (matches section)
<i>Data Input</i>	currentUserId to filter Match rows

Match List Item

- Partner name and profile photo
- Partner department and year
- Request link tracking text (from which request the match was created)
- Request type indicator badge
- openChatButton — bypasses all navigation logic and directly loads the ChatScreen for their shared Conversation

<i>Component</i>	BottomNavigationBar — Persistent Shell Container Layout
<i>Behavior</i>	Always visible when inside the main app (HomeScreen, RequestsScreen, ChatsScreen, ProfileScreen)
Tab 1 — Home	House icon. Routes to HomeScreen. Filled/colored when active, outline when inactive.
Tab 2 — Requests	List icon. Routes to RequestsScreen. Maintains last active sub-tab state on return.
Tab 3 — Chats	Chat bubble icon. Routes to ChatsScreen. Real-time unread count badge. Badge clears when all chats are read.
Tab 4 — Profile	Person icon. Routes to ProfileScreen.

Every screen-to-screen route in the application. Arrows (→) indicate navigation destinations triggered by user actions or system logic.

SplashScreen

- LoginScreen — if no session exists
- ProfileSetupScreen — if session but profile is incomplete
- HomeScreen — if session and profile is complete

LoginScreen

- HomeScreen — on successful authentication
- SignupScreen — on tap signup link
- ForgotPasswordScreen — on tap forgot password link

SignupScreen

- EmailVerificationScreen — on successful account creation

EmailVerificationScreen

- ProfileSetupScreen — on correct OTP verification

ProfileSetupScreen

- HomeScreen — on Save & Continue

HomeScreen (Tab 1)

- CreateRequestScreen — tap FAB
- RequestDetailsScreen — tap open request card
- PartnerProfileScreen — tap partner card
- NotificationsScreen — tap notification icon
- MatchesScreen — tap see all matches

RequestsScreen (Tab 2)

- CreateRequestScreen — tap FAB
- RequestDetailsScreen — tap on Browse tab item
- MyRequestDetailsScreen — tap on My Requests tab item

MyRequestDetailsScreen

- PartnerProfileScreen — view responder profile
- ChatScreen — after accepting a response (Open Chat button)

ChatScreen

- PartnerProfileScreen — tap partner name in top bar

ChatsScreen (Tab 3)

- ChatScreen — tap any conversation item

ProfileScreen (Tab 4)

- EditProfileScreen — tap editProfileButton
- SettingsScreen — tap settingsButton
- LoginScreen — on logout confirmation
- MatchesScreen — tap matches section
- NotificationsScreen — tap notification area

PartnerProfileScreen

- ChatScreen — if already matched (Open Chat button)
- ReportUserScreen — from overflow menu or footer

NotificationsScreen

- MyRequestDetailsScreen — tap REQUEST_RECEIVED notification
- ChatScreen — tap NEW_MESSAGE or REQUEST_ACCEPTED notification
- RequestsScreen (My Requests Tab) — tap REQUEST_REJECTED notification
- ProfileScreen — tap STREAK_MILESTONE notification

SettingsScreen

- LoginScreen — on delete account confirmation

These rules apply universally across every screen in the application and must be implemented without exception.

Loading States

- Every screen that loads async data must display polished grey skeleton placeholder cards/blocks.
- Blank white or empty screens during loading are strictly forbidden.
- Skeleton layout must mirror the shape and structure of the real content it represents.

Empty States

- Any list with zero records must display an illustrative image (non-default, contextually relevant).
- Accompanying helper text must be contextually precise and actionable (e.g. "No open requests found. Create one!").
- Empty states must not show a plain 'No data' text without visual treatment.

Error Handling

- Network failures and database errors must trigger a Snackbar alert immediately.
- Snackbar message must specify what failed (not a generic error).
- Snackbar must include an active Retry action button.

Button Interaction Controls

- Any button that triggers a remote operation must disable its tap listener on tap.
- A circular progress indicator must render inside or near the button until the transaction completes.
- On success or failure, button must return to interactive state (enabled or error state).

Confirmation Overlays

- Destructive or irreversible workflows must require a confirmation modal.
- Affected flows: Logout, Delete Account, Close Request.
- Modal must contain exactly two clearly labeled actions: Cancel and Confirm.
- No destructive action should execute without an explicit Confirm tap.

Navigation Standards

- All stack-pushed secondary screens must display an explicit back arrow in the top bar.
- Back arrow must route cleanly back to the calling screen without side effects.
- Disabled UI controls must be visually greyed out.
- Disabled controls must display clarifying tooltip text or inline hint text explaining why they are disabled.
- Disabled controls must NEVER show generic system exception bubbles or crash dialogs.

CampusBuddy — Full Application Description

Platform: Android (Kotlin) / Jetpack Compose · All 23 Sections · Version 1.0

This document is the complete specification for CampusBuddy. It covers all 19 screens, 8 data models, 7 enumerations, 5 authentication flows, profile setup, 14 core screen workflows with step-by-step execution logic, UI state management for every component, a complete screen-to-screen navigation map, streak and reliability calculation algorithms, and 6 universal UI rules applicable to every screen. This specification is intended as a complete handoff document for Android developers implementing CampusBuddy in Kotlin with Jetpack Compose.